

Padrões de Projeto (Proxy)

QXD0068 - Reuso de Software

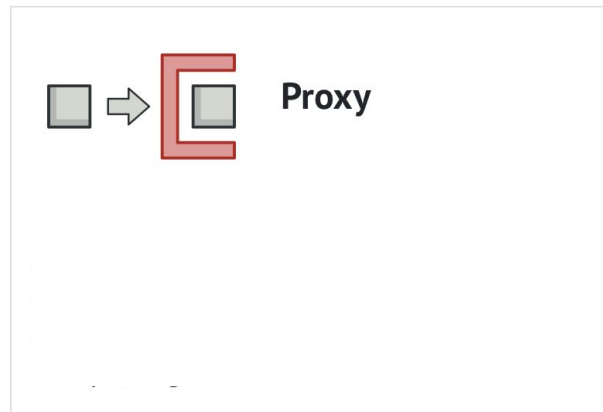
Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Proxy
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação

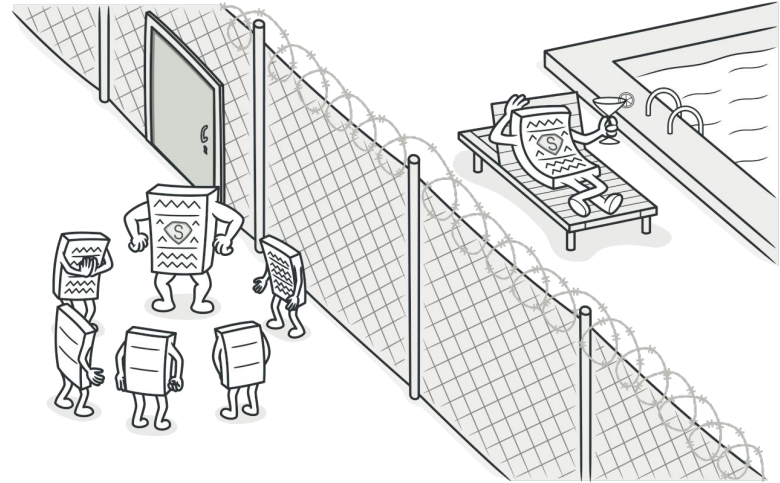


Introdução

- O Proxy é um padrão de projeto estrutural que permite criar um substituto (ou intermediário) para outro objeto, controlando o acesso ao objeto real.
- Ele atua como um “representante” ou “guarda de acesso”, podendo interceptar, modificar ou adiar as chamadas ao objeto original — sem que o cliente perceba a diferença.
- Esse padrão é muito útil quando o objeto real:
 - É pesado para ser instanciado (demora, consome recursos, etc.);
 - Está em outro processo ou servidor;
 - Precisa de controle de acesso ou cache de resultados.

Padrão Proxy

- O Proxy serve para intermediar a comunicação entre o cliente e o objeto real, adicionando uma camada extra de controle.
- Com ele, é possível executar ações antes ou depois do acesso ao objeto real — como validações, logs, cache, controle remoto ou inicialização preguiçosa — sem alterar o código do serviço original.

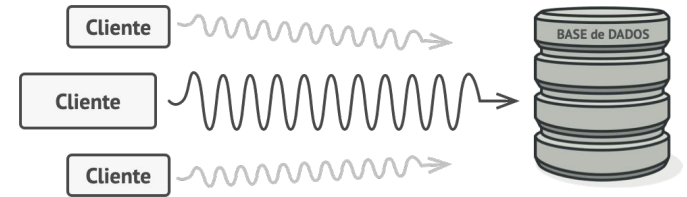


Problema

- Por que eu iria querer controlar o acesso a um objeto? Aqui está um exemplo: você tem um objeto grande que consome muitos recursos do sistema. Você precisa dele de tempos em tempos, mas não sempre.
- Problema resolvido pelo padrão Proxy
 - Solicitações para bases de dados podem ser bem lentas.
- Você poderia implementar uma inicialização preguiçosa: criar esse objeto apenas quando ele é realmente necessário. Todos os clientes do objeto teriam que executar algum código adiado de inicialização. Infelizmente, isso provavelmente resultaria em muito código duplicado.
- Em um mundo ideal, gostaríamos que você colocasse esse código diretamente dentro da classe do nosso objeto, mas isso nem sempre é possível. Por exemplo, a classe pode fazer parte de uma biblioteca fechada de terceiros.

Problema

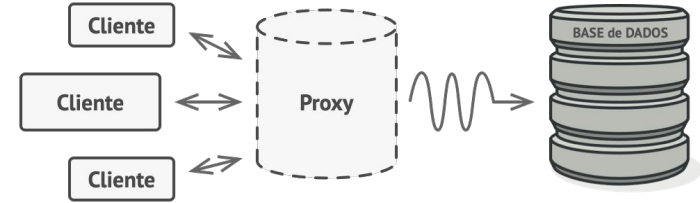
- **Em muitos sistemas, há objetos que:**
 - São custosos para carregar (como conexões com banco de dados, imagens grandes, ou bibliotecas externas);
 - Precisam de controle de acesso (por exemplo, apenas usuários autenticados);
 - Ou estão em locais remotos (como servidores externos).
- Implementar diretamente todas essas verificações no cliente ou no objeto principal gera código duplicado e baixa coesão.
- Além disso, às vezes não é possível alterar a classe original, como quando ela pertence a uma biblioteca de terceiros.



Uma classe que baixa vídeos do YouTube diretamente. Cada solicitação faz um novo download, mesmo que o vídeo já tenha sido baixado — desperdiçando tempo e banda.

Solução

- **O padrão Proxy propõe criar uma classe intermediária (proxy) que:**
 - Implementa a mesma interface da classe de serviço original.
 - Encapsula uma instância do serviço real.
 - Controla quando e como o serviço real será chamado.
- **Essa classe pode adicionar comportamentos como:**
 - Cache de resultados para evitar repetições;
 - Controle de acesso antes de repassar a solicitação;
 - Inicialização preguiçosa, criando o objeto real apenas quando necessário;
 - Registro de logs e monitoramento das operações.
- O cliente interage apenas com o proxy, sem precisar saber se está lidando com o objeto real ou com o substituto.

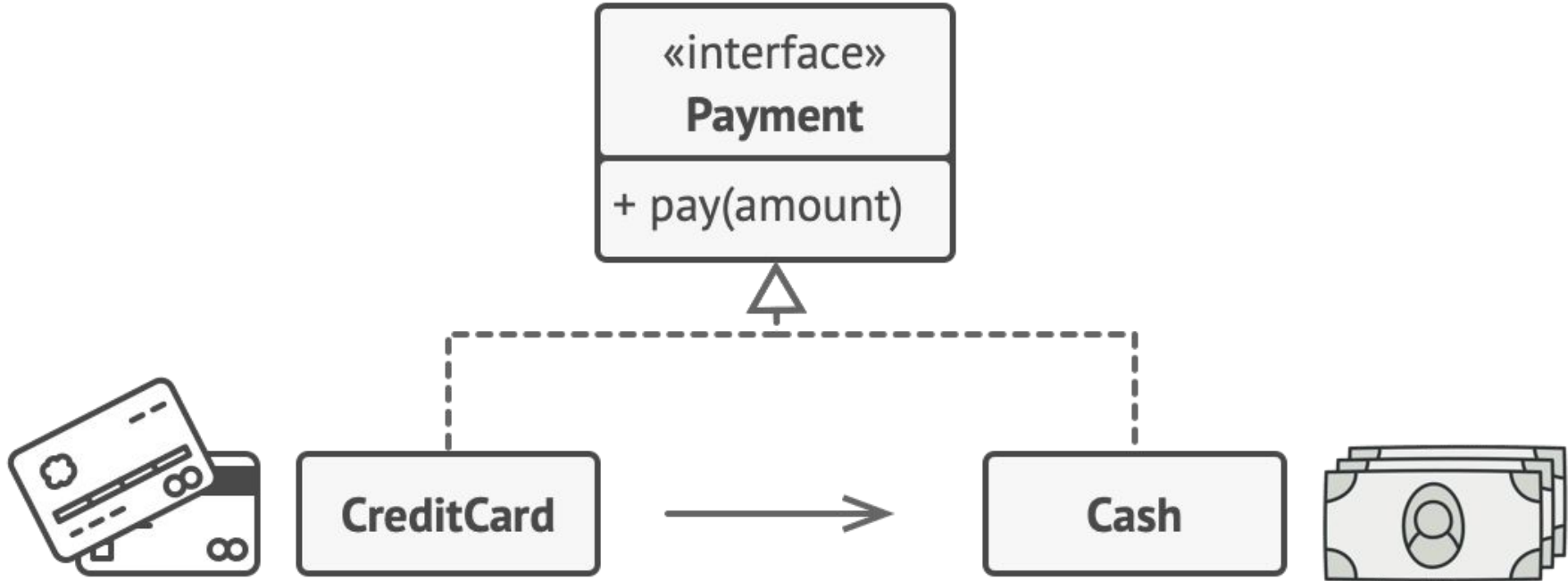


O proxy se disfarça de objeto de base de dados. Ele pode lidar com inicializações preguiçosas e caches de resultados sem que o cliente ou a base de dados fiquem sabendo.

Analogia do Mundo Real

- Um cartão de crédito é um ótimo exemplo de proxy para uma conta bancária.
- O cartão e a conta têm a mesma interface funcional — ambos permitem pagar algo — mas o cartão intermedia o acesso ao dinheiro, adicionando camadas de segurança e conveniência:
 - O cliente não precisa carregar dinheiro físico (inicialização preguiçosa);
 - O comerciante recebe o valor sem contato direto com o dinheiro (controle de acesso);
 - A transação é registrada eletronicamente (registro e monitoramento).

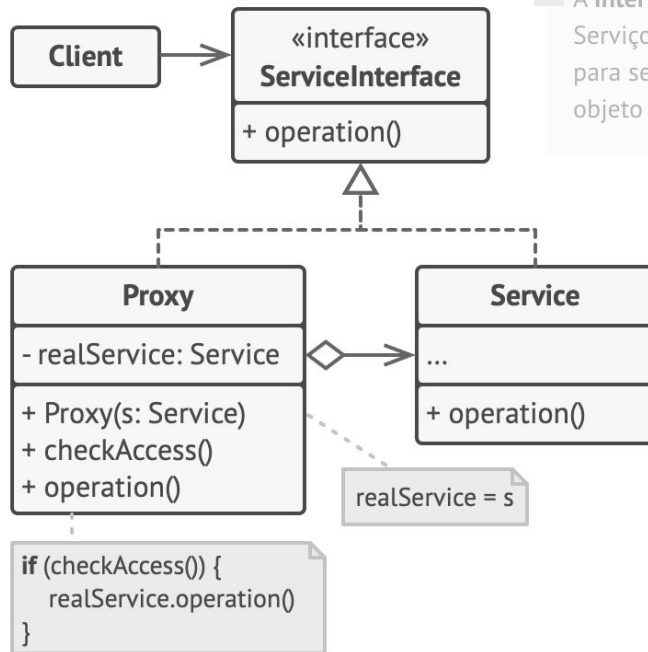
Analogia do Mundo Real



Estrutura do Padrão

- **Interface do Serviço**
 - Define as operações que tanto o serviço real quanto o proxy devem oferecer.
- **Serviço Real (Real Subject)**
 - Classe principal que executa a lógica central do sistema. Pode ser pesada ou de difícil acesso.
- **Proxy (Substituto)**
 - Implementa a mesma interface do serviço real, mantendo uma referência a ele.
 - Decide quando e como delegar as chamadas, adicionando comportamento extra (cache, logs, etc.).
- **Cliente**
 - Interage com o proxy da mesma forma que interagiria com o serviço real — sem precisar saber qual está sendo usado.

4 O **Cliente** deve trabalhar tanto com os serviços e proxies através da mesma interface. Dessa forma você pode passar uma proxy para qualquer código que espera um objeto do serviço.



1 A **Interface do Serviço** declara a interface do Serviço. O proxy deve seguir essa interface para ser capaz de se disfarçar como um objeto do serviço.

3 A classe **Proxy** tem um campo de referência que aponta para um objeto do serviço. Após o proxy finalizar seu processamento (por exemplo: inicialização preguiçosa, acesso, acessar controle, colocar em cache, etc.), ele passa o pedido para o objeto do serviço.

Geralmente os proxies gerenciam todo o ciclo de vida dos seus objetos de serviço.

2 O **Serviço** é uma classe que fornece alguma lógica de negócio útil.

```
// Interface comum
interface FileAccess {
    fun openFile(filename: String)
}

// Serviço real
class RealFileAccess : FileAccess {
    override fun openFile(filename: String) {
        println("Abrindo o arquivo: $filename")
    }
}

// Proxy que controla o acesso
class SecureFileProxy(private val userRole: String) : FileAccess {
    private val realAccess = RealFileAccess()

    override fun openFile(filename: String) {
        if (userRole == "admin") {
            println("Acesso autorizado para $userRole.")
            realAccess.openFile(filename)
        } else {
            println("Acesso negado para $userRole. Permissão insuficiente.")
        }
    }
}

// Cliente
fun main() {
    val adminProxy = SecureFileProxy("admin")
    val userProxy = SecureFileProxy("guest")

    adminProxy.openFile("dados_confidenciais.txt") // permitido
    userProxy.openFile("dados_confidenciais.txt") // bloqueado
}
```

Exemplo em Kotlin

Um sistema de arquivos tem uma classe `FileAccess` que abre arquivos. Queremos criar um **Proxy** que **verifique permissões** antes de liberar o acesso.

```
from abc import ABC, abstractmethod
import time

# Interface comum
class YouTubeService(ABC):
    @abstractmethod
    def get_video(self, video_id: str) -> str:
        pass

# Serviço real - simula uma operação custosa
class YouTubeAPI(YouTubeService):
    def get_video(self, video_id: str) -> str:
        print(f"Baixando vídeo '{video_id}' do YouTube...")
        time.sleep(2) # simula demora da rede
        return f"Conteúdo do vídeo {video_id}"

# Proxy que adiciona cache
class CachedYouTubeProxy(YouTubeService):
    def __init__(self, real_service: YouTubeService):
        self._real_service = real_service
        self._cache = {}

    def get_video(self, video_id: str) -> str:
        if video_id not in self._cache:
            print("Cache ausente. Chamando o serviço real...")
            self._cache[video_id] = self._real_service.get_video(video_id)
        else:
            print("Retornando vídeo do cache.")
        return self._cache[video_id]

# Cliente
if __name__ == "__main__":
    service = YouTubeAPI()
    proxy = CachedYouTubeProxy(service)

    print(proxy.get_video("abc123")) # Primeira chamada (busca real)
    print(proxy.get_video("abc123")) # Segunda chamada (cache)
```

Exemplo em Python

Temos um serviço que busca vídeos de uma API (simulada). O **Proxy** adiciona **cache** para evitar requisições repetidas e melhorar a performance.

Aplicabilidade

- **Use o Proxy quando:**
 - **Inicialização preguiçosa (Virtual Proxy):** O objeto real é criado apenas quando necessário (ex.: carregar imagens sob demanda).
 - **Controle de acesso (Protection Proxy):** Restringe o acesso a certos objetos (ex.: controle de permissões em sistemas críticos).
 - **Proxy remoto (Remote Proxy):** Representa objetos que estão em outro servidor ou processo (ex.: RMI, gRPC).
 - **Proxy de cache (Caching Proxy):** Armazena e reutiliza resultados de chamadas anteriores.
 - **Proxy de registro (Logging Proxy):** Grava logs de chamadas e transações sem modificar o código original.
 - **Referência inteligente (Smart Reference Proxy):** Monitora o uso de objetos, liberando recursos quando não há mais referências ativas.

Prós e Contras

- **Vantagens**

- Permite controlar o acesso ao objeto real sem modificá-lo.
- Facilita adiar a criação de objetos custosos.
- Possibilita adicionar cache, logs, autenticação e outras camadas extras.
- Segue o princípio aberto/fechado: novos proxies podem ser criados sem alterar o código existente.

- **Desvantagens**

- Aumenta a complexidade estrutural (mais classes e camadas).
- Pode introduzir atrasos adicionais na comunicação.
- Pode dificultar a depuração se houver várias camadas de proxies.



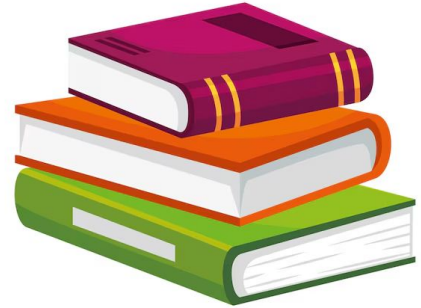
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado!

Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

